Lab16–DatabaseDesignandQueries

Name: D.Rishwanth

Roll.no:2503A52395

Batch:14

TaskDescrption-1:(DesignDatabaseSchemaforStudentManagement System)

Prompt:

GenerateSQLCREATETABLEstatementsforaStudentManagementSystemwith three tables: Students, Courses, and Enrollments. Include appropriate data types, primary keys, foreign keys, and constraints. Ensure the database design follows 1NF, 2NF, and3NF. Keep the SQLcode simple and compatible withMySQL.

Code:

```
--Create Database
CREATEDATABASEStudentManagement;

--Select Database
USEStudentManagement;

-- Create Students Table
CREATETABLEStudents(
    student_id INT PRIMARY KEY,
    nameVARCHAR(100)NOTNULL,
    email VARCHAR(100) UNIQUE
);

-- Create Courses Table
CREATETABLECourses(
    course_idINTPRIMARYKEY,
```

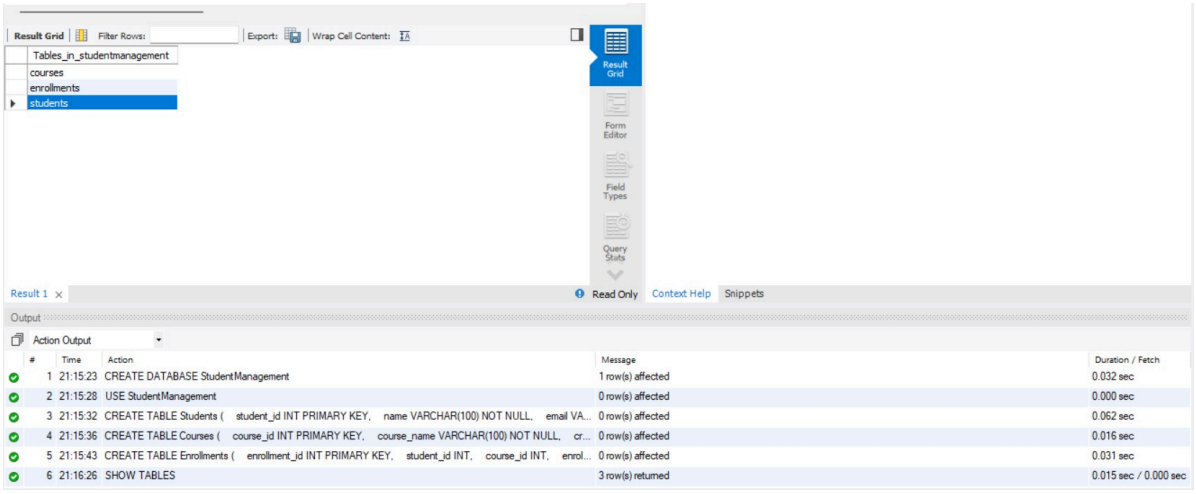
```
    course_nameVARCHAR(100)NOTNULL,
    credits INT
);

-- Create Enrollments Table
CREATETABLEEnrollments(
    enrollment_idINTPRIMARYKEY,
    student_id INT,
    course_id INT,
    enrollment_dateDATE,

    FOREIGN KEY (student_id)
    REFERENCESStudents(student_id),

    FOREIGN KEY (course_id)
    REFERENCESCourses(course_id)
);
```

Output:



Justification:

Createdthreetables:Students,Courses,andEnrollmentsusingSQL.Primarykeys uniquelyidentifyrecords, while foreignkeys establishrelationships between tables. The database follows 1NF, 2NF, and 3NF to reduce data redundancyand maintain data integrity.

TaskDescription-2:(InsertSampleRecords)

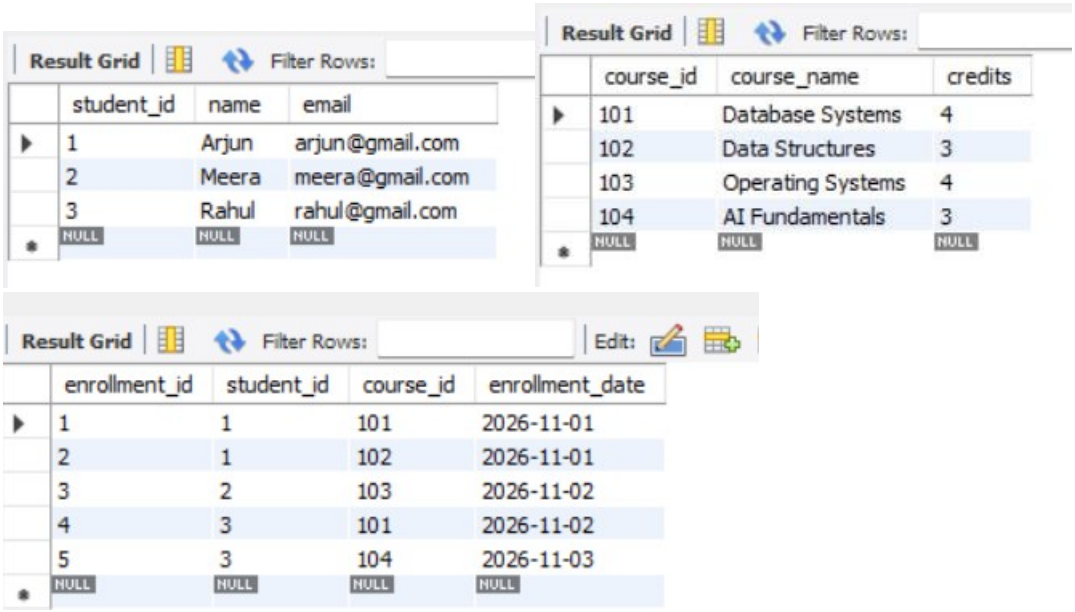
Prompt:

GenerateSQLINSERTstatementstoinsertatleast3students,4courses,and5 enrollment records into the Students, Courses, and Enrollments tables.Also generate SELECT queries to verify the inserted records.

Code:

```
INSERTINTOStudentsVALUES
(1,'Arjun','arjun@gmail.com'),
(2,'Meera','meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERTINTOCoursesVALUES
(101,'DatabaseSystems',4),
(102,'DataStructures',3),
(103,'OperatingSystems',4),
(104, 'AI Fundamentals', 3);
INSERTINTOEnrollmentsVALUES (1,
1, 101, '2026-11-01'),
(2,1,102,'2026-11-01'),
(3,2,103,'2026-11-02'),
(4,3,101,'2026-11-02'),
(5,3,104,'2026-11-03');
```

Output:



Result Grid | Filter Rows:

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com
NULL	NULL	NULL

Result Grid | Filter Rows:

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3
NULL	NULL	NULL

Result Grid | Filter Rows: | Edit:

enrollment_id	student_id	course_id	enrollment_date
1	1	101	2026-11-01
2	1	102	2026-11-01
3	2	103	2026-11-02
4	3	101	2026-11-02
5	3	104	2026-11-03
NULL	NULL	NULL	NULL

Justification:

InsertedsamplerecordsintoStudents,Courses,andEnrollmentstablesusing INSERT statements. Used SELECT queries to verify the inserted records successfully.

TaskDescription-3:(PerformCRUDOperations) Instructions:

Prompt:

GenerateSQLqueriestoperformCRUDoperationsontheStudentManagement Database. Retrieve all courses, update a student's email, delete an enrollment record, and verify the changes using SELECT statements.

Code:

```
SELECT*FROMCourses;


UPDATEStudents

SETemail='arjun_new@gmail.com'W

HERE student_id = 1;


DELETEFROMEnrollments

WHERE enrollment_id = 5;
```

SELECT*FROMStudents;

SELECT*FROMEnrollments; Output:

```
67 •  SELECT * FROM Students;
```

Result Grid | Filter Rows: | Edit

student_id	name	email
1	Arjun	arjun_new@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com
NULL	NULL	NULL

```
69 •  SELECT * FROM Enrollments;
```

Result Grid | Filter Rows: | Edit:

enrollment_id	student_id	course_id	enrollment_date
1	1	101	2026-11-01
2	1	102	2026-11-01
3	2	103	2026-11-02
4	3	101	2026-11-02
NULL	NULL	NULL	NULL

Justification:

PerformedCRUDoperationsusingSELECT,UPDATE,andDELETEstatements.
VerifiedtheupdatedemailanddeletedenrollmentrecordusingSELECTqueries.

TaskDescription-4:(ExecuteAggregateQueries)

Prompt:

GenerateSQLaggregatequeriestocountthenumberofstudentsenrolledineach course and display courses having more than one student enrolled using GROUP BY, COUNT(), and HAVING clauses.

Code:

```
SELECT c.course_name,
    COUNT(e.student_id)ASstudent_count
FROM Courses c
JOINEnrollmentse
ON c.course_id= e.course_id
```

GROUPBYc.course_name;

SELECTc.course_name

FROM Courses c

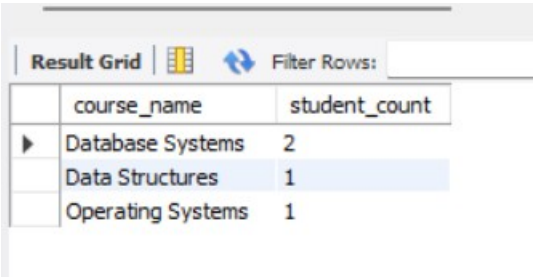
JOINEnrollmentse

ON c.course_id = e.course_id

GROUP BY c.course_name

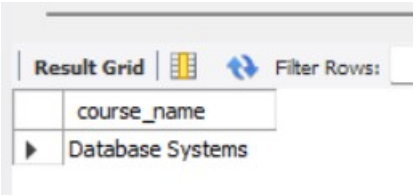
HAVINGCOUNT(e.student_id)>1;

Output:



Result Grid | Filter Rows:

course_name	student_count
Database Systems	2
Data Structures	1
Operating Systems	1



Result Grid | Filter Rows:

course_name
Database Systems

Justification:

UsedaggregatefunctionsCOUNT(),GROUPBY,andHAVINGtocountstudents enrolled in each course and identify courses with more than one student.

TaskDescription-5:(QueryOptimization)

Prompt:

AnalyzethegivenSQLsubqueryandrewriteitusingJOINsforbetterquery performance. Ensure both queries return the same results.

Code:

```
SELECT*FROMStudents
WHERE student_id IN (
    SELECT student_id FROM
    Enrollments
    WHEREcourse_id=101
```

);

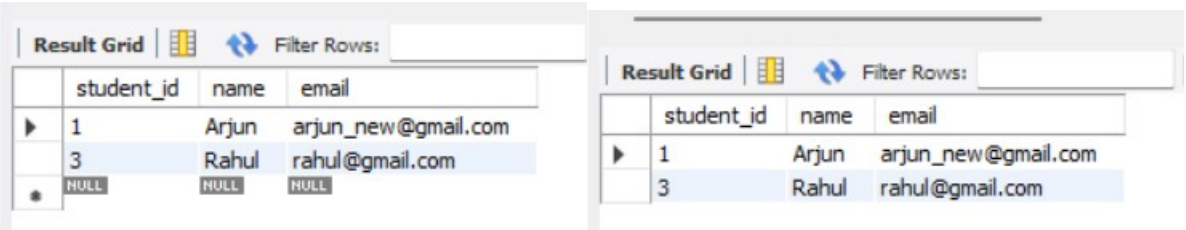
SELECTs.*

FROM Students s

JOINEnrollmentse

ONs.student_id=e.student_id

WHERE e.course_id = 101; Output:



student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com
NULL	NULL	NULL

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

Justification:

RewrotetheoriginalsubqueryusingJOINtoretrievestudentsenrolledinaspecific course. Compared both queries to verify that they produce the same results and improve query efficiency.